面向对象程序设计

C++新特性（续）



★C++与C基础知识点差异提炼

西南交通大学
Southwest Jiaotong University

C++的新特性

1. C++数据的输入输出 (1.4.3节)
2. 名字空间namespace (1.4.2节)
3. 引入新的数据类型(2.1.2节、10章、11章)
4. 带默认参数的函数 (2.5.3节)
5. 内联函数 (2.5.4节)
6. 函数重载 (2.5.5节)
7. 内存管理 (3.3.5节)
8. const (3.3.7节)
9. C++引用 (3.4节)
10. 异常处理 (12章)

★C++新特性：函数重载(overload)(P69)

一个或多个函数，名字相同，然而参数个数或参数类型不相同，这叫做函数的重载。

以下三个函数是重载关系：

```
① double Max(double f1,double f2) {     }
② int Max(int n1,int n2) {     }
③ int Max(int n1,int n2,int n3) {     }
```

区别继承机制：函数重写(override)

```
Max_double_double
Max_int_int
Max_int_int_int
```

- 函数重载使得函数命名变得简单。
- 编译器根据调用语句中的实参的个数和类型判断应该调用哪个函数。

```
Max(3.4,2.5);        //调用①
Max(2,4);            //调用②
Max(1,2,3);          //调用③
Max(3,2.4);          //error，二义性
```

★C++新特性：函数重载(over load)

西南交通大学 Southwest Jiaotong University

排序：实现2、3、4个整型变量值由小到大排列。

```
void ascend(int &a, int &b)
 {
     if(a>b)  swap(a,b); //系统自带swap
}
 void ascend(int &a, int &b, int &c)
{
  if(a>b)
     swap(a,b);
  if(a>c)
        swap(a,c); //使a值最小
  if(b>c)
        swap(b,c); //使b<c
}
```

```
void ascend(int &a,int &b, int &c, int &d)
{
      if(a>b)
            swap(a,b);
      if(a>c)
            swap(a,c);
      if(a>d)
            swap(a,d);      //以上使a值最小
      ascend(b,c,d);      //使b, c, d升序排列
}
```

★C++新特性：函数重载(overload)

西南交通大学 Southwest Jiaotong University

函数重载应注意的问题:

- 返回值类型不同不能作为函数重载的条件。
- 函数重载调用时，可能出现不可分辨错误。

例：

```
int f_a(float x) { return (int)x; }
int f_a(double x) { return (int)x; }
void main()
{   cout<<f_a(5)<<endl;        }        //??
```

此外，

```
void f_b(int x);
void f_b(int &x);  //也出现不可分辨错误
```

★C++新特性：带默认参数的函数（函数的缺省参数）（P65）

- C++允许在函数的原型(reference declaration)或函数的定义(definition declaration)中给一个或多个参数指定初值（默认值/缺省值）。
- 定义函数的时候可以让最右边的连续若干个参数有初值（默认值/缺省值），那么调用函数的时候，若相应位置不写参数，参数的值就是缺省值。

```
void func( int x1, int x2 = 2, int x3 = 3)
{
}
func(10 ) ; //等效于 func(10,2,3)
func(10,8); //等效于 func(10,8,3)
```

实参与形参结合是从左至右顺序进行的，因此缺省的参数必须放在形参表列中的最右端，否则错误！

```
func(10, , 8) ; //×，只能最右边的连续若干个参数缺省
```

C++新特性： 带默认参数的函数

例： 错误缺省参数实例。

```
int f_b(int x,int y=9);        //原型

int f_b(int x,int y=9)
  {
      return x+y;
   }    //定义
```

注 意： 不能在函数原型和定义中同时指定缺省参数!

正确的做法是：

- 仅在原型声明中给出缺省参数

```
int f_b(int x,int y=9);
int f_b(int x,int y)
{ return x+y; }
```

- 仅在定义中给出缺省参数

```
int f_b(int x,int y);
int f_b(int x,int y=9)
{ return x+y; }
```

★C++新特性： 带默认参数的函数

- 在指定了缺省值的参数右边不能出现没有指定缺省值的参数

例： void f1(float a,int b,int c=0,char d);

//参数d没有缺省值，错误!

- 函数缺省值和函数重载可能不可分辨

例：

```
void f() { }
int f(int x=5) { return x+1; }
```

//函数调用f(); 不可分辨，错误!

函数的缺省参数

- 函数参数可缺省的目的在于提高程序的可扩充性。
- 如果某个写好的函数要添加新的参数，而原先那些调用该函数的语句，未必需要使用新增的参数，那么为了避免对原先那些函数调用语句的修改，就可以使用缺省参数。



C++新特性：带默认参数的函数

西南交通大学 Southwest Jiaotong University

单选题：下列C++函数定义**错误**的是（ ）。



A `Fun(int x=0, int y=0, int z=0)`

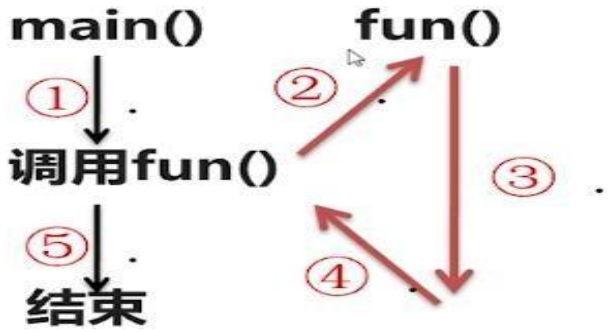
B `Fun(int x, int y=0, int z=0)`

C `Fun(int x=0, int y=0, int z)`

D `Fun(int x, int y, int z)`

★C++新特性：内联函数(inline)(P67)

西南交通大学 Southwest Jiaotong University

普通函数	内联函数
	为了减少函数调用的开销，引入了内联函数机制。编译器处理对内联函数的调用语句时，是将整个函数的代码插入到调用语句处，而不会产生调用函数的语句。

宏定义	内联函数
#define MUTIL（a） （a*a） 1. 宏不能访问对象私有成员 2. 宏不能进行类型检查 3. 宏定义容易产生二义性	内联函数实现宏定义的替换功能，进行常规类型检查，以及其他可以预见的状态。

★C++新特性：内联函数(inline)

西南交通大学 Southwest Jiaotong University

- C++允许在编译时将所调用函数的代码嵌入到主调函数中，称为内联函数(inline function)，以提高代码的执行效率。定义内联函数使用关键字inline。

```
//内联函数定义
inline int max(int a,int b,int c)
{
        int m;
        if(b>a) a=b;
        if(c>a) a=c;
        m=a;
        return m;
}
int main(int argc, char *argv[])
{
        int i=20,j=20,k=30,m;  m=max(i,j,k);
        cout<<"max="<<m<<endl;
}
```

```
//内联函数编译后的效果
int main()
{
        int i=10,j=20,k=30,m;
        int a,b,c;
        a=i;b=j;c=k;
        if(b>a) a=b;
        if(c>a) a=c;
        m=a;
        cout<<"max="<<m<<endl;
        return 0;
}
```

★C++新特性：内联函数(inline)

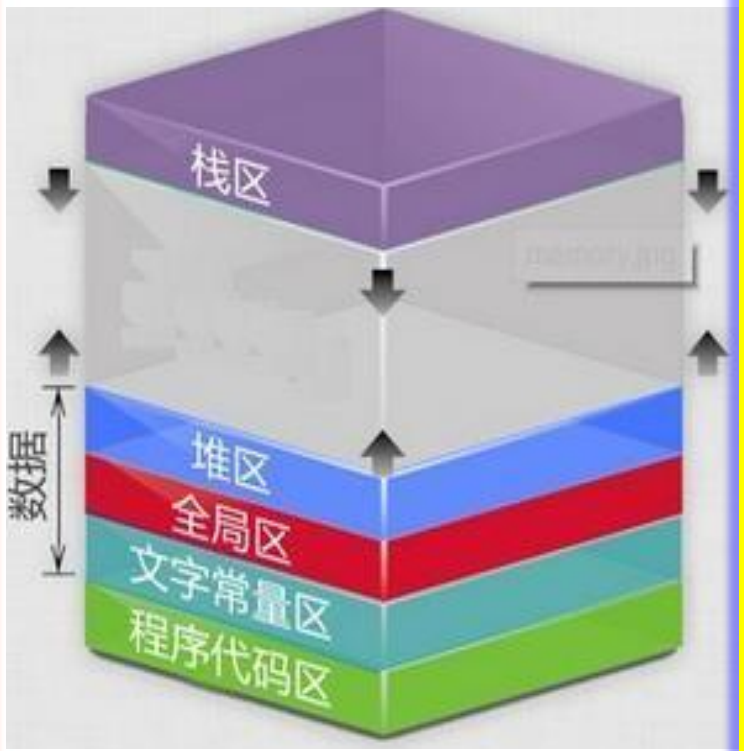
内联函数（inline）注意事项

- 内联函数在编译时由编译器决定是否进行内联优化操作
- 内联逻辑简单，一般代码不超过5行，调用频繁的函数建议使用内联
- 复杂的内联函数：如递归，循环之类的语句编译器会忽略。



C++新特性9：内存管理new、delete(P112)

认识内存分区



在C++中，内存分成5个区，他们分别是栈、堆、全局/静态存储区、常量存储区和代码区。

- 栈：由编译器在需要的时候分配，里面的变量通常是局部变量、函数参数等。
- 堆：由new(malloc)分配的内存块，分配后空间的释放不由编译器完成，而由我们应用程序去控制。
- 全局/静态存储区：全局变量和静态变量被分配到同一块内存中，在以前的C++堆栈中，全局变量又分为初始化的和未初始化的，在C++里面没有这个区分了，他们共同占用同一块内存区。
- 常量存储区：这是一块比较特殊的存储区，他们里面存放的是常量，不允许修改。

★C++新特性：内存管理new、delete


```
#include <iomanip>
using namespace std;
int k=1;
int Max(int a,int b)//在调用此函数时参数从右往左开始压栈
{
    return a>b?a:b;
}
int main()
{
    int i=1;
    int *j= new int[2]; //一般不确定需要多大空间的时候用
    static int m=1;
    char *n="hello";
    void *k1=n;


    cout<<"栈区地址为:"<<hex<<&i<<endl;
    cout<<"堆区地址为:"<<hex<<j<<endl;
    delete []j;//及时释放
    cout<<"全局变量地址为:"<<hex<<&k<<endl;
    cout<<"静态变量地址为:"<<hex<<&m<<endl;
    cout<<"文字常量区地址为:"<<hex<<k1<<endl;
    cout<<"代码区地址为:"<<(void *)&Max<<endl;
    return 1;
}
```

内存分区

```
栈区地址为:0x6efe90
堆区地址为:0x7b10f0
全局变量地址为:0x482004
静态变量地址为:0x482008
文字常量区地址为:0x489000
代码区地址为:0x401500
```

★C++新特性： 内存管理new、delete

1. C语言中内存的动态分配（malloc、calloc、realloc、free函数）

① malloc()用法

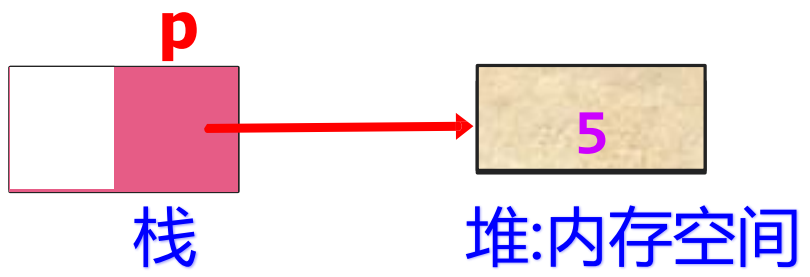
```
char * str=(char *)mallloc(80);        //分配80个字节内存空间
string *str1;                          //可否 malloc string内存空间?
```

② free()用法

```
free(str);
```

2. C++语言中新的内存动态分配处理模式（new和delete运算符）

```
int * p = new int(4);
*p = 5;
delete p;
```



★C++新特性：内存管理new、delete

西南交通大学 Southwest Jiaotong University

new & delete的配合使用

应用程序一般使用malloc，realloc，new等从堆中分配到一块内存，分配的内存使用完后，程序必须负责调用相应的free或delete释放该内存块。

- new

```
int    *i = new int;            // (1) 没有初始值
int    *j = new int(100);       // (2) 初始值为100
string *iStr = new string[3];
//(3)分配块空间，例如数组空间分配:具有3个字符串
```

- delete

```
delete i;        // (1) 释放单个变量所占用的内存
delete j;        // (2) 释放单个变量所占用的内存
delete []iStr;// (3) 释放数组所占用的内存
```

★C++新特性： 内存管理new、delete

针对任意类型指针P，如string *P;

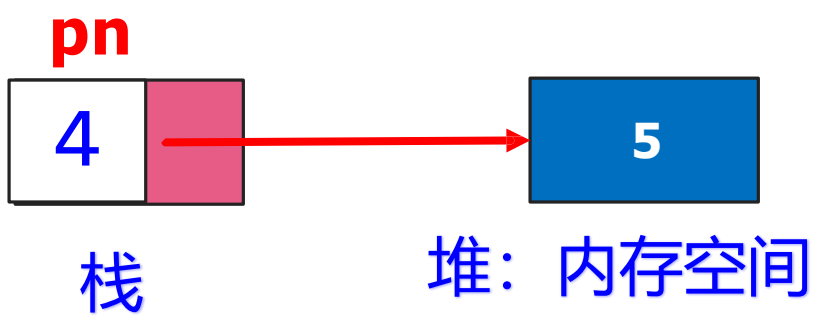
3. 用new运算符实现动态内存分配

分配一个指针变量空间: P = new T;

T是任意类型名，P是类型为T * 的指针。

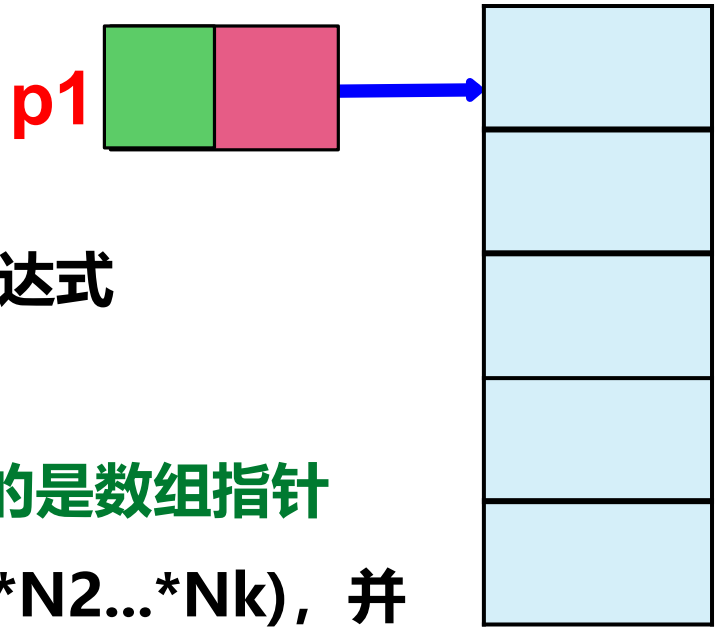
动态分配出一片大小为sizeof(T)字节的内存空间，并且将该内存空间的起始地址赋值给P。如:

```
int * pn;
pn = new int;
*pn = 5;
```



★C++新特性：内存管理new、delete

- 分配一个一维数组：T *P = new T[N1];
- 分配一个多维数组：T (*P)[N2]...[Nk] = new T[N1][N2]...[Nk];



- T: 任意类型名
- P: 类型为T *的指针
- N1,N2,...Nk: 要分配的数组第k维元素的个数，可以是整型表达式

```
int * p1=new int[10];
int (*p2) [3][4]=new int[2][3][4]; //多维数组，注意定义的是数组指针
```

- 动态分配出一片大小为sizeof(T*N)字节的内存空间（N=N1*N2...*Nk），并且将该内存空间的起始地址赋值给P。

★C++新特性： 内存管理new、delete

西南交通大学 Southwest Jiaotong University

4. 用delete运算符释放动态分配的内存

用new动态分配的内存空间，一定要用delete运算符进行释放

```
delete 指针；//该指针必须指向new出来的空间
```

```
int * p = new int;
* p = 5;
delete p;
delete p; //导致异常，一片空间不能被delete多次
```



★C++新特性：内存管理new、delete

内存动态分配

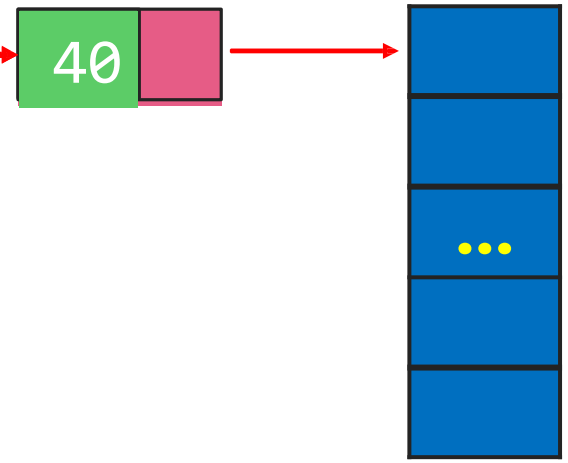
```
T  (*P)  [N] = new T[M][N];
int (*p)[5]=new int[3][5];
int(*p)[3][5]=new int [2][3][5];
```

5. 用delete运算符释放动态分配的数组

用delete释放动态分配的数组，要加[ ]

```
delete [] 指针；  //该指针必须指向new出来的数组
```

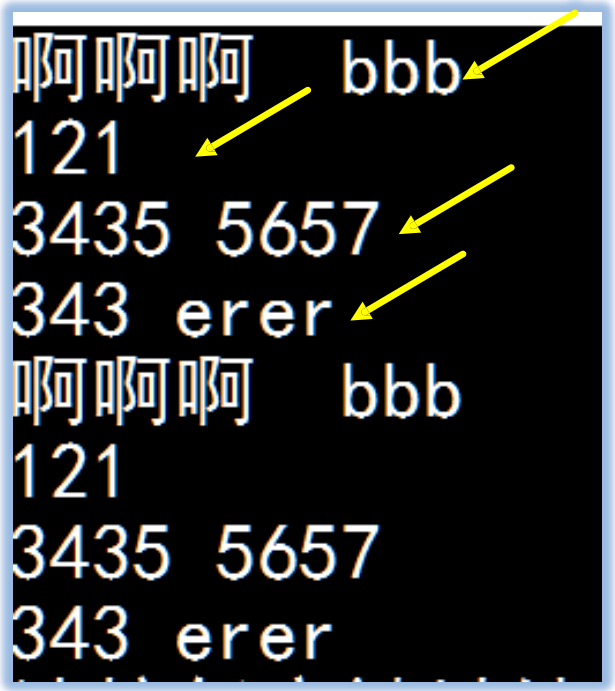
```
int * p = new int[10];
p[0] = 1;
delete [] p;
```



★C++新特性： 内存管理new、delete

```
int main()
{
    string (*s1)[2]=new string[2][2];
    for(int i=0;i<2;i++)
        for(int j=0;j<2;j++)
            getline(cin,s1[i][j]);
    for(int i=0;i<2;i++)
        for(int j=0;j<2;j++)
            cout<<s1[i][j]<<"\n";
    delete []s1;  //why not:  delete [][]s1;

    system("pause");
    return 1;
}
```



★C++新特性：内存管理new、delete

new/delete 和malloc/free注意问题

① new/delete必须配对使用，malloc/free也一样。

② 如果用free释放new创建的动态对象，该对象因无法执行析构函数可能导致程序出错。

③ 如果用delete释放malloc申请的动态内存，理论上讲程序不会出错，但是该程序的可读性很差。

```
#include <iostream>
using namespace std;
class myclass
{
  public:
    ~myclass()
    { cout << "Goodbye" << endl; }
};
```

```
int main()
{
    myclass *p = new myclass;
    //free(p);
    delete p;
    return 0;
}
```

★C++新特性：异常处理(P382)

10



异常处理：什么情况下会用

1. 程序运行中总难免因为细节性问题，出现不正常反映
   - (1) 数组元素的下标超界、访问NULL指针
   - (2) 意外的非法输入，除数为0
   - (3) 动态内存分配new需要的存储空间太大
   - (4) 程序不健壮，调用别人程序
2. 引起这些异常情况的原因
   - (1) 代码质量不高，存在BUG
   - (2) 输入数据不符合要求
   - (3) 程序的算法设计时考虑不周到
   - (4) ……

```
void main()
{
    string equation="1+1=";
    cout<<equation[4];
}
```

C:\Windows\system32\cmd.e

Microsoft Visual C++ Debug Library

Debug Assertion Failed!

Program: E:\Program\DD\Debug\DD.exe
File: d:\program files (x86)\microsoft visual studio 2010\vc\include\xstring
Line: 1441

Expression: string subscript out of range

For information on how your program can cause an assertion failure, see the Visual C++ documentation on asserts.

(Press Retry to debug the application)

★C++新特性：异常处理(try、throw、catch)

西南交通大学 Southwest Jiaotong University

3. 我们总希望在发生异常情况时，及时处理
   - 不只是简单地终止程序运行
   - 反馈异常情况：哪一段代码发生的? 什么异常?
   - 对程序运行中已发生的事情做些处理：取消对输入信息及文件的改动、释放已经申请的系统资源

4. 通常的做法

① 在预计会发生异常的地方，加入相应的代码，但这种做法并不总是适用的

```
……  //对文件A进行了相关的操作
fun(arg, ……);  //可能发生异常
```

★C++新特性：异常处理(try、throw、catch)

西南交通大学 Southwest Jiaotong University

② caller该如何知道fun(arg, ……)是否发生异常

- 没有发生异常，可以继续执行
- 发生异常，应该在结束程序运行前还原对文件A的操作

③ fun(arg, ……)是别人已经开发好的代码

- fun(arg, ……)的编写者不知道其他人会如何使用这个函数
- fun(arg, ……)会出现在表达式中，通过返回值的方式区分是否发生异常
- 不符合编写程序的习惯
- 可能发生多种异常，通过返回值判断也很麻烦

★C++新特性：异常处理(try、throw、catch)

西南交通大学 Southwest Jiaotong University

异常处理（try、throw、catch）模式

① C++中，try与catch用于实现异常的处理。

② C++处理异常的机制由3个部分组成

- 检查(try)
- 抛出(throw)
- 捕捉(catch)

③ 处理流程：把需要检查的语句放在try块中，throw用来当出现异常时发出一个异常信息，而catch根据异常信息类型来捕捉异常信息，如果捕捉到了异常信息，就处理它。

★C++新特性：异常处理(try、throw、catch)

西南交通大学 Southwest Jiaotong University

try结构

try：用来检测可能出现异常的语句

throw：抛出异常

catch：用来捕捉异常信息，如果捕捉到了异常信息，就处理它

```
void g()
{
    try
    {
        f();
        throw 异常;
    }
    catch(int x)
    {
    }
    catch(char s)
    {
    }
}
```

★C++新特性：异常处理(try、throw、catch)

西南交通大学 Southwest Jiaotong University

用try、catch处理异常

计算：m/n

```
double m ,n;
cin >> m >> n;
try
{
    if( n == 0)
      throw -1.0; //抛出异常
    if( m == 0)
      throw -1;
    else
        cout << m / n << endl;
}
```

```
catch(double d)
    { cout << "catch(double) " << d <<endl;}
catch(int e)
    { cout << "catch(int) " << e << endl;}
catch(…)
    { cout << "catch(…) " << "Err" << endl;}
cout << "finished" << endl;
```



★Java 异常处理

```
public class TestException {
    public static void main(String[] args) {
        int a = 1;
        int b = 0;
        try { // try监控区域
                if (b == 0) throw new ArithmeticException(); // 通过throw语句抛出异常
                System.out.println("a/b的值是: " + a / b);
            System.out.println("this will not be printed!");
        }
        catch (ArithmeticException e) { // catch捕捉异常
                System.out.println("程序出现异常，变量b不能为0! ");
        }
        System.out.println("程序正常结束。");
    }
}
```

★Python 异常处理

```
a=10
b=0
try:
    c=a/b
    print c
except ZeroDivisionError,e:
    print e.message
print "done"
```



★C++新特性：异常处理(try、throw、catch)

西南交通大学 Southwest Jiaotong University

异常处理注意事项

① 首先把可能出现异常的、需要检查的语句或程序段放在try后面的花括号中。被检测的函数必须放在try块中，否则不起作用。

② try块和catch块作为一个整体出现，catch块是try-catch结构中的一部分，必须紧跟在try块之后，不能单独使用，在二者之间也不能插入其他语句。但是 在一个try-catch结构中，可以只有try块而无catch块。

程序开始运行后，按正常的顺序执行到try块，开始执行try块中花括号内的语句。如果在执行try块内的语句过程中没有发生异常，则catch子句不起作用， 流程转到catch子句后面的语句继续执行。一个try-catch结构中只能有一个try块，但却可以有多个catch块，以便与不同的异常信息匹配。

★C++新特性：异常处理(try、throw、catch)

西南交通大学
Southwest Jiaotong University

③ try和catch块中必须有用花括号括起来的复合语句，即使花括号内只有一个语句，也不能省略花括号。

④ 如果在执行try块内的语句（包括其所调用的函数）过程中发生异常，则throw运算符抛出一个异常信息。throw抛出异常信息后，首先在本函数中寻找与之匹配的catch，如果找不到，流程立即离开本函数，转到其上一级的函数。throw抛出什么样的数据由程序设计者自定，可以是任何类型的数据。

⑤ 这个异常信息提供给try-catch结构，系统会寻找与之匹配的catch子句。

⑥ 在进行异常处理后，程序并不会自动终止，继续执行catch子句后面的语句。

★C++新特性：异常处理(try、throw、catch)

西南交通大学 Southwest Jiaotong University

⑦ catch只检查所捕获异常信息的类型，而不检查它们的值。因此如果需要 检测多个不同的异常信息，应当由throw抛出不同类型的异常信息。异常信息可以是C++系统预定义的标准类型，也可以是用户自定义的类型(如结构体或类)。

⑧ 如果在catch子句中没有指定异常信息的类型，而用了删节号“...”， 则表示它可以捕捉任何类型的异常信息。

```
catch(...)
{
    cout<<"OK"<<endl;
}
```

try-catch结构可以与throw出现在同一个函数中，也可以不在同一函数中。当throw抛出异常信息后，首先在本函数中寻找与之匹配的catch，如果在本函数中无try catch结构或找不到与之匹配的catch，就转到离开出现异常最近的try-catch结构去处理。

⑨ 如果throw抛出的异常信息找不到与之匹配的catch块，那么系统就会调用一个系统函数terminate，使程序终止运行。

★给出程序运行结果

```
void f1()
{
    try
    {
        f2();
    }
    catch(char)
    {
        cout<<"char: ok"<<endl;
    }
}
```

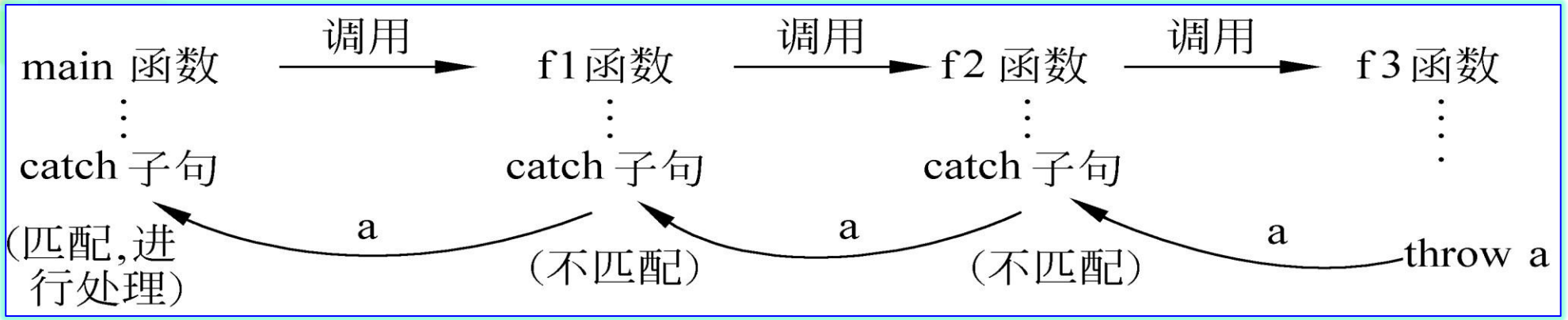
```
void f2()
{
    try
    {
        f3();
    }
    catch(int)
    {
        cout<<"int: ok"<<endl;
    }
}
```



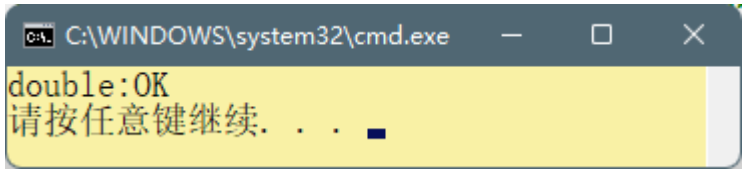
```
void f3()
{
    double a=0;
    try
    {
    throw a;
    }
    catch(float)
    {
        cout<<"float: ok"<<endl;
    }
}
```

```
int main()
{
    try
    {
        f1();
    }
    catch(double )
    {
        cout<<"double: ok"<<endl;
    }
    return 1;
}
```

★C++新特性：异常处理(try、throw、catch)



运行结果:



在主函数中捕获异常

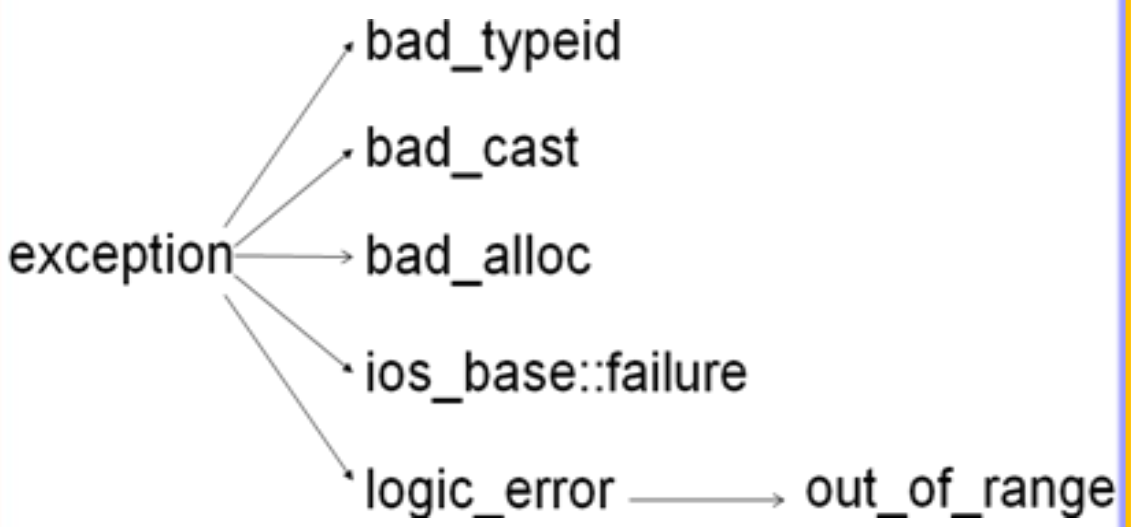
执行主函数中最后一个语句时的输出

如果一个函数在执行的过程中，抛出的异常在本函数内就被catch块捕获并处理了，那么该异常就不会抛给这个函数的调用者(也称上一层的函数)；如果异常在本函数中没被处理，就会被抛给上一层的函数。

★C++新特性：异常处理(try、throw、catch)

C++标准异常类

C++标准库中有一些类代表异常，这些类都是从exception类派生而来。常用的几个异常类如下：



- bad_cast在用dynamic_cast进行从多态基类对象(或引用)，到派生类的引用的强制类型转换时，若转换是不安全的，则会抛出此异常。
- bad_alloc在用new运算符进行动态内存分配时，若没有足够的内存，则会引发此异常。
- out_of_range用vector或string的at成员函数根据下标访问元素时，若下标越界，就会抛出此异常。

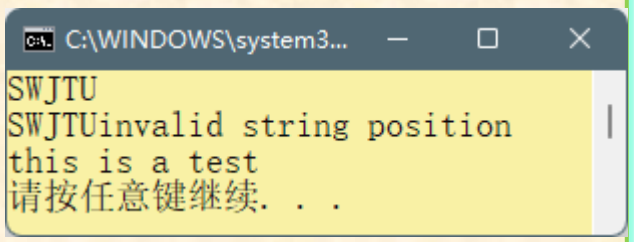
★C++新特性：异常处理(try、throw、catch)

标准异常类处理

```
#include<iostream>
#include<string>
#include<stdexcept>
using namespace std;
int main() {
string  a_str;
int i=0;
cin>>a_str;
try
{ while(1)
  {
   cout<<a_str.at(i++); //at成员函数会抛出out_of_range类型异常
  }
}
```

```
catch(out_of_range e1)
{
    cout<<e1.what()<<endl;
}

    cout<<"this is a test "<<endl;
    return 0;
}
```



★读程序写出执行结果


```
#include<iostream>
#include<string>
#include<stdexcept>
using namespace std;
int main()
{
    string a_str;
    cin>>a_str;
    try
    {
        throw string("error");
    }
    catch(string)
    {
        cout<<"Error"<<endl;
    }
    cout<<"this is a test "<<endl;
    return 0;
}
```



★作业

1. 打印如下图形到屏幕，打印行数n由键盘输入。

```
    a
   abc
  abcba
 abcdcba
```

要求：用cin和cout实现输入/输出。

2. 用new算符建立n元double型数组，输入n个实数存于数组，由大到小排序 后输出数组元素，最后用delete删除数组。程序要求先输入n值。

3. 编写C++程序，输入若干整数，到0停止。

(1)用先入先出法建立带附加头结点的单链表。

(2)输出单链表；　(3)删除负数结点；

(4)输出删除负数结点后的单链表。

要求：用cin和cout实现输入/输出；

用new和delete建立和删除结点。

★作业

4. 用带有2个引用型形参的函数求取一个n元整型一维数组的最大值和第二大值元素下标。主函数输入n个整数，调用该函数，然后输出最大值和第二大值。要求n值用const定义。

5. 定义两个重载函数min，分别求2个和3个整数的最小值。主函数先输入2个整数，输出其中的最小值；然后输入3个整数，输出其中的最小值。

6. 定义两个重载函数find(a, n, x), find(a, i, n, x)分别表示数组a从下标0开始和下标i开始按下标加1顺序查找x，直到下标n结束。要求函数返回a[j]==x并且j>=0(或i)的最小下标j值。查找失败，返回-1 。主函数定义m元数组，输入m个元素，然后输入x ，用以上两个重载函数实现统计x在数组中的出现次数。要求m用const定义。

★作业

7. 编程题：以下是程序框架，请根据代码中的描述，完成函数的定义，注意有两个show()函数，每个都使用了默认参数，请尽可能使用引用和const参数。

```
struct STRING
{
        char *str;//字符指针指向字符串
        int ct;//字符串的长度
};
//定义函数 set  show
void set(   )//此处需添加函数参数
{
        int i=0;
        for(;pstr[i];i++);
        str.ct=i;
        str.str=new char[i+1];
        //此处按要求3 补充代码
}
void show() //补充参数和执行代码
void show() //补充参数和执行代码
```

```
int main()
{
        STRING str;
        char str_1[]="hello world";
        set(str,str_1);
        //要求：(1) 第一个参数是引用
        // (2) 分配空间给STRING中str
        // (3) 把str_1中的数据复制到str中
        // (4) 计算ct=字符串大小
        show(str);              //输出str字符串
        show(str,3);            //输出str前三个字符
        show(str_1);            //输 出 str_1
        show(str_1,3);          //输出str_1前三个字符
        show("finished");
        return 0;
}
```

★作业

8. 编写程序实现功能：输入两个数，第一个数为被除数，第二个数为除数， 若 除数为0，则抛出一个异常并要求重新输入两个数；若除数不为 0，则计算并显示结果。

